EQUILIBRIUM

A Multi-Agent AI Planning Assistant

Project Report on

Design and Implementation of an Orchestrated LLM Agent Swarm for Autonomous Goal Decomposition and Tool-Augmented Task Execution

In partial fulfilment of the requirements for the award of the

Degree of

Bachelor of Technology in

Computer Science and Engineering

Submitted by

Justin — Architecture, Agents & Backend

Anushka — Product, Design & UX

Department of Computer Science and Engineering
School of Engineering and Technology
Academic Year 2025–2026

Declaration

We, Justin and Anushka, hereby declare that this project report entitled “Equilibrium — A Multi-Agent AI Planning Assistant: Design and Implementation of an Orchestrated LLM Agent Swarm for Autonomous Goal Decomposition and Tool-Augmented Task Execution”, submitted by us in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work carried out by us.

The system, its multi-agent architecture, the orchestration and critic pipeline, the model-fallback layer, and the front-end interface described herein were designed and implemented by us. This work has not been submitted previously, in part or in full, for the award of any other degree, diploma, or similar title or recognition.

Place: —

Date: July 2026

Justin (Architecture, Agents & Backend)

Anushka (Product, Design & UX)

Certificate

This is to certify that the project report entitled “Equilibrium — A Multi-Agent AI Planning Assistant”, submitted by Justin and Anushka, students of Bachelor of Technology in Computer Science and Engineering, is an original contribution to existing knowledge and a faithful record of work carried out by them under guidance and supervision.

The candidates have successfully engineered a working end-to-end system comprising a React front end, an Express/WebSocket backend, and an orchestrated multi-agent reasoning engine backed by a resilient large-language-model fallback chain. To the best of our knowledge, this work has not been submitted in part or full for any degree or diploma to this or any other institution.

Project Guide

Department of Computer Science and Engineering

Acknowledgement

We take this opportunity to express our profound sense of gratitude to everyone who supported us throughout this project. It would have been difficult to complete this work without their encouragement and technical guidance.

We are grateful to our faculty guide for continual direction and for helping us refine both the engineering and the presentation of this system. We also thank the Head of the Department and the faculty of the Department of Computer Science and Engineering for their regular encouragement and for providing the environment in which this project could be built and tested.

We extend our thanks to the open developer community behind the tools and hosted model APIs that made rapid experimentation possible, and to our families and friends for their steady support during the course of this work.

Justin

Anushka

Table of Contents

1. Abstract

Large language models are individually capable but unreliable when asked to carry a complex, multi-step goal from start to finish. A single model prompted to “plan a three-day trip” or “research and write a market report” tends to hallucinate structure, skip steps, emit placeholder filler, and fail completely when the hosting API is rate-limited or momentarily unavailable. Equilibrium addresses this by treating a goal not as one prompt but as a coordination problem solved by a small team of specialised agents.

Equilibrium (delivered under the product name GreenLeaf) is a full-stack, multi-agent planning assistant. A user states a goal in plain English through a React interface; the request travels over a WebSocket to an Express backend where an orchestrator LLM decomposes it into the smallest ordered set of concrete subtasks and routes each one to the specialist best equipped for it — a Researcher with web-search access, a Writer that produces and saves documents, an Analyst that runs code and calls external APIs, or a Generalist with access to every tool. The specialists share memory so that a “gather” subtask automatically feeds a “use it” subtask.

A tool-calling loop lets each specialist reason, invoke real tools, observe results, and continue until its subtask is complete. A Synthesiser then composes the individual results into a single Markdown answer, and a Critic model reviews that answer against the original goal, demanding at most one revision before anything reaches the user. Underpinning all of this is a resilient model layer: every LLM call automatically falls back through a chain of NVIDIA-hosted models on rate limits, outages, empty responses, or unavailable model identifiers, with per-model cooldowns so a temporarily throttled model is skipped rather than retried. The result is a system that turns an unreliable single call into a coordinated, observable, and fault-tolerant pipeline whose progress streams live into the interface and whose written outputs are downloadable directly from the conversation.

2. Introduction

2.1 Background

The assumption that a single language-model invocation can reliably execute a complex objective is a critical flaw in naïve LLM applications. Real goals — planning a trip, producing a researched document, analysing a dataset — are inherently multi-step: they require gathering current information, reasoning over it, producing an artefact, and checking the result. A monolithic prompt collapses all of these into one generation pass, where the model must simultaneously plan, research, write, and self-verify. In practice it does none of them well: it invents facts it should have looked up, produces stubs where it should produce content, and offers no recovery path when the underlying API fails.

The multi-agent paradigm decomposes this difficulty. Rather than one model doing everything, an orchestrator divides the goal and delegates to role-specialised agents, each given a narrow remit and a matching set of tools. This mirrors how human teams operate — a researcher gathers, a writer drafts, an analyst computes, and a reviewer critiques — and it makes the whole process observable and correctable at each stage. Equilibrium is an implementation of exactly this paradigm as a production-shaped, full-stack application.

2.2 Problem Statement

Autonomous goal-completion systems built on hosted language models face three coupled problems. First, decomposition: a goal must be split into the right subtasks and each assigned to an agent whose capabilities match, without producing redundant or vague steps. Second, grounded execution: agents must use real tools (web search, file writing, code execution, HTTP calls) rather than fabricating results, and must build on each other's outputs instead of repeating work. Third, resilience and quality: hosted models are rate-limited and occasionally unavailable, reasoning models can consume their entire token budget and return nothing, and any of them can emit placeholder filler — so the system needs both a fault-tolerant model layer and a review stage that rejects weak answers. There is a clear need for an architecture that unifies orchestration, tool-augmented specialist execution, model-level fault tolerance, and answer criticism into one coherent, observable pipeline.

2.3 Objectives of the Study

1. **To design an orchestrator** that decomposes a natural-language goal into the smallest set of concrete, ordered subtasks and assigns each to the appropriate specialist role.
2. **To implement role-specialised agents** (Researcher, Writer, Analyst, Generalist) with strictly scoped tool authorisations and a shared-memory tool-calling loop that grounds every step in real tool output.
3. **To build a resilient model layer** that transparently falls back across a chain of hosted LLMs on rate limits, outages, empty responses, and unavailable models, with per-model cooldowns.
4. **To add a synthesis-and-critic stage** that composes specialist results into one answer and enforces a quality gate permitting a single revision.

5. **To deliver a live, usable interface** that streams every agent step over WebSocket and exposes written artefacts as downloads.

2.4 Scope of the Study

The scope of this project is the design, implementation, and qualitative evaluation of an orchestrated multi-agent system for general goal completion using hosted language models and a fixed set of tools (web search, file writing, code execution, and HTTP API calls). It covers goal decomposition, role-based tool-calling execution, model-level fault tolerance, answer synthesis and criticism, real-time progress streaming, and authenticated multi-user sessions. Training or fine-tuning of language models, large-scale quantitative benchmarking, and deployment hardening for high-concurrency production traffic are beyond the scope of this study.

2.5 Significance of the Study

This work demonstrates a practical, reproducible pattern for turning unreliable single-model calls into a coordinated and fault-tolerant system. The architecture shows how orchestration, scoped tool use, model fallback, and answer criticism combine to raise completion quality and robustness without model retraining. The findings are directly transferable to intelligent assistants, automated research and reporting tools, and decision-support systems, and they contribute to the growing body of applied work on agentic AI and reliable tool-using language-model applications.

3. Literature Review

3.1 Orchestrator–Worker Multi-Agent Systems

A dominant pattern in agentic AI is the orchestrator–worker (or manager–specialist) topology, in which one coordinating model plans and delegates while subordinate agents execute. The central benefit is separation of concerns: the orchestrator reasons about what needs to happen and in what order, while each worker focuses on a narrow, well-tooled task. Equilibrium adopts this pattern directly — its Planner produces an ordered JSON list of subtasks, each tagged with a role — and constrains plans to the smallest useful set (typically two to three subtasks) to avoid the redundant or vague steps that inflate cost and latency in less disciplined designs.

3.2 Tool-Augmented Agents and the ReAct Loop

Tool-augmented reasoning, popularised by the ReAct family of methods, interleaves model reasoning with tool invocation: the model proposes an action, observes the result, and repeats until the task is done. This grounds generation in real data and is the foundation of modern function-calling agents. Equilibrium implements this as a bounded tool-calling loop in which each specialist may call only the tools its role authorises, and every tool result is appended to a shared context so later steps build on earlier evidence rather than re-deriving it.

3.3 Role Specialisation and Scoped Tool Authorisation

Research on agent teams shows that giving each agent a distinct role and a restricted toolset improves reliability and reduces unsafe or off-task behaviour compared with a single agent holding every capability. Narrow tool scopes also shrink the decision space at each step, which measurably improves tool-selection accuracy. Equilibrium encodes this as an explicit authorisation matrix: the Researcher may only search, the Writer may only write files, the Analyst may only run code and call APIs, and only the Generalist holds all four tools.

3.4 Self-Reflection and Critic Models

A recurring finding is that language-model output improves when a separate reviewing pass evaluates it against the original objective and requests targeted revisions — the “reflection” or “critic” pattern. Unbounded self-revision, however, risks loops and runaway cost. Equilibrium takes the pragmatic middle path: a dedicated Critic model judges the synthesised answer once and may demand a single revision, and it is engineered to fail open (approve) on any error so it can only improve, never block, a run.

3.5 Reliability, Routing, and Model Fallback

Because hosted model endpoints impose per-model rate limits and occasionally return errors, empty completions, or retire model identifiers, production agent systems increasingly rely on routing and fallback layers that redirect a failed call to an alternative model. Equilibrium contributes a concrete, benchmarked implementation of this idea: a primary model chosen by tool-calling benchmark, an ordered fallback chain, per-model cooldown tracking so throttled models are skipped, and model-specific request

tuning (extra token headroom for reasoning models, disabled inline reasoning for the primary) to prevent empty answers.

3.6 Summary

The literature converges on four ingredients for reliable agentic systems: an orchestrator that decomposes and delegates, tool-augmented workers grounded through a ReAct-style loop, role specialisation with scoped tools, and a reflective critic for quality control — all of which are only as dependable as the model-serving layer beneath them. Equilibrium is a synthesis of these ideas into a single working system, and its distinctive contribution is combining them with a rigorously engineered model-fallback layer so that the whole pipeline degrades gracefully rather than failing outright when any individual model call goes wrong.

4. Methodology

4.1 Research and System Design

Equilibrium is built as an end-to-end, event-driven application rather than a script. The design is layered: a React/Vite front end for interaction and live progress; an Express server that terminates a WebSocket connection and owns the agent run; an agent core comprising a planner, a role registry, an executor, a synthesiser, and a critic; and a shared model-access module that fronts all LLM calls. This separation lets each concern — interface, transport, orchestration, execution, and model resilience — be reasoned about and tested independently.

4.2 System Architecture

Figure 1 shows the overall architecture. The browser client renders the goal, streams progress, and exposes written files as download chips. A single WebSocket session carries the goal in and progress and answers out. On the backend, the Planner turns the goal into subtasks, the specialists execute them against real tools, the Synthesiser composes the result, and the Critic reviews it. All model calls route through the NVIDIA NIM fallback chain, and tools reach out to the web, the file system, and a code runner.

Figure 1: Equilibrium Multi-Agent System Architecture

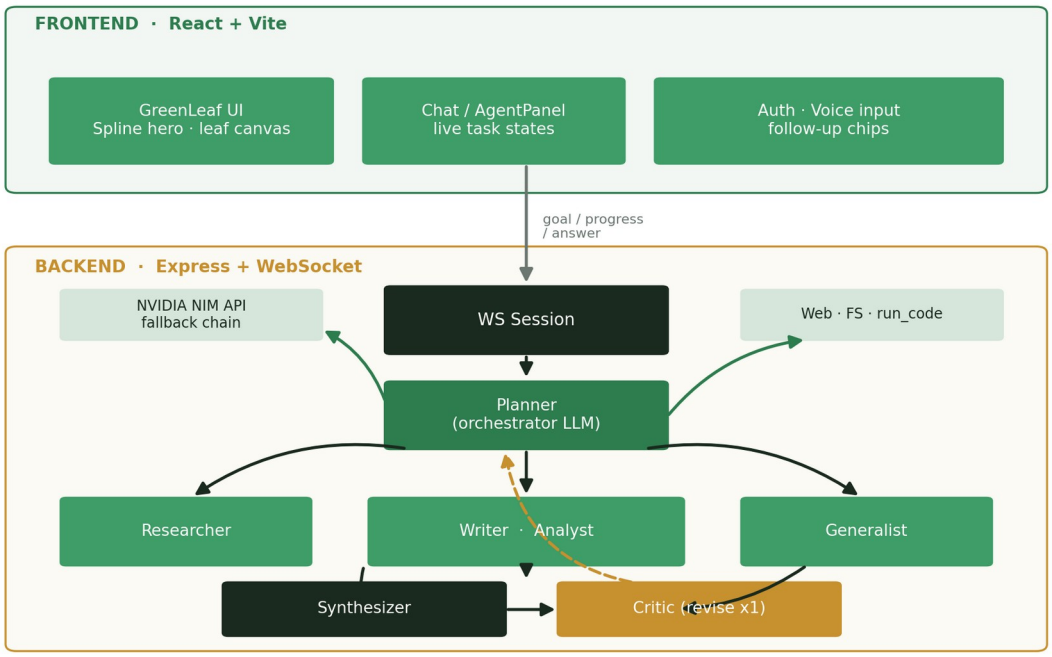

Figure 1: Equilibrium Multi-Agent System Architecture

4.3 The Orchestrator (Planner)

The Planner is an LLM given a strict orchestration prompt. It divides the user's goal into the smallest set of concrete, ordered subtasks — usually two or three — and assigns each to one of four roles. It is told

that the team shares memory, so a gathering subtask can feed a subsequent using subtask, and it is instructed to match the role to the tools each subtask needs (research before writing, analyse before reporting). The Planner must respond only with a JSON array; the implementation strips code fences, salvages the outermost array if the model wraps it in prose, and normalises each task. If the planner is rate-limited or returns unparseable output, the system falls back to a single generalist task carrying the original goal, so a planning hiccup never crashes a run.

4.4 Role-Specialised Agents

Four specialists share one executor but are differentiated by a role-specific system prompt and a whitelist of permitted tools. Table 1 records this authorisation matrix, and Figure 4a visualises it.

Role	Permitted tools	Responsibility
Researcher	web_search	Gather accurate, current information and report concrete findings with sources.
Writer	write_file	Turn gathered material into a complete, well-structured document saved to the workspace.
Analyst	run_code, call_api	Perform computation, parsing, and external data retrieval; show real numbers.
Generalist	all four tools	Complete mixed tasks end-to-end using whichever tools fit.

Table 1: Specialist roles and their scoped tool authorisations

Every specialist also inherits shared hard rules: never output placeholder text, treat teammates' earlier results as source material rather than re-doing their work, keep tool arguments as valid JSON, and finish with a short note stating exactly what was produced.

4.5 The Tool-Calling Execution Loop

The executor drives each subtask through a bounded reason–act–observe loop. The specialist model is offered its authorised tools; when it proposes a tool call, the executor validates the arguments as a plain JSON object, executes the corresponding function, and appends the result to the running message history that constitutes shared memory. The loop continues until the model returns a final textual answer or a step budget is reached. Robustness is built in: if a single request exceeds a model's context limit the executor trims context rather than failing, and rate-limit errors are handled by the model layer beneath it.

Four tools are available. Web search attempts Brave (if a key is configured), then a keyless DuckDuckGo HTML scrape, then DuckDuckGo's instant-answer API, then Wikipedia — a cascade that returns useful results without requiring any paid key. File writing is confined to a workspace directory, rejecting absolute paths and "../" escapes because the path originates from the model. Code execution writes the model's JavaScript to a temporary file and runs it under a timeout and output cap. The API tool performs arbitrary HTTP requests with optional headers and JSON bodies.

4.6 Synthesis and the Critic Loop

Once all subtasks complete, the Synthesiser composes their results into a single, clean Markdown answer that contains the actual findings and content produced — not a description of the process — and references any files written. The Critic then judges that answer against the original goal for completeness, accuracy, on-topic focus, and freedom from placeholders, responding only with JSON: either an approval or a short piece of feedback triggering exactly one revision. The Critic runs on a fast small-model chain because latency matters more than depth for a judgement call, and it fails open: on any error it approves, so it can only improve a run, never break it.

Figure 2: Agent Execution Sequence (Plan → Execute → Critique)

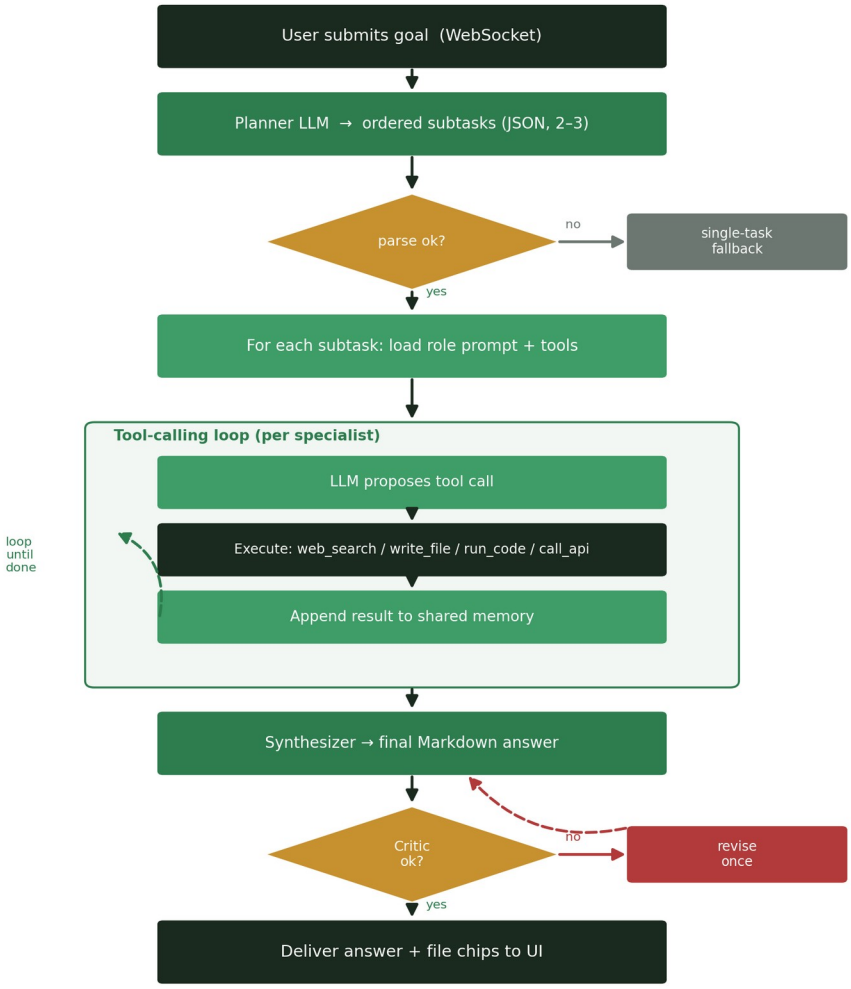

Figure 2: Agent execution sequence — plan, execute in a tool loop, synthesise, and critique

4.7 The Model Fallback Layer

All LLM access is centralised in one module that fronts NVIDIA's OpenAI-compatible NIM endpoint. Every call is issued against a primary model and, on failure, retried transparently down an ordered chain — because each hosted model has its own quota bucket, falling back keeps the agent working. Rate-limit (HTTP 429), model-unavailable (404), and empty-response conditions advance the chain; a request-too-

large error instead signals the caller to shrink context. Rate-limited models are recorded in a cooldown map and skipped until their quota resets, and if every model is briefly throttled the layer waits out a short per-minute reset rather than surfacing an error. The primary model was selected by benchmarking tool-calling behaviour on the account's catalogue; model-specific tuning gives reasoning models extra token headroom and disables the primary's inline reasoning so its whole budget goes to the answer. Figure 3 summarises the chain and the benchmark that informed it.

Figure 3a: NVIDIA NIM Model Fallback Chain

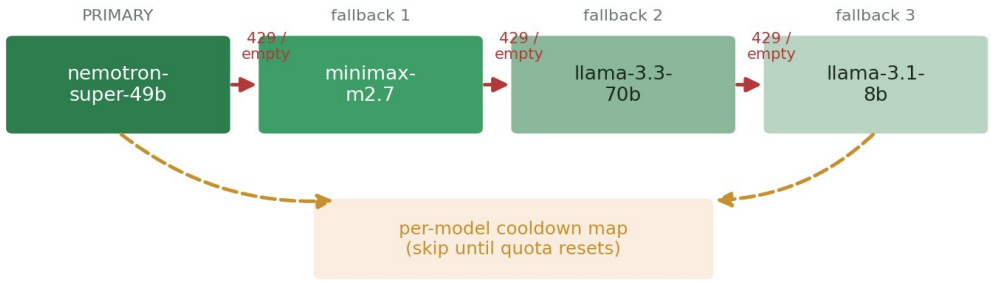

Figure 3b: Tool-Calling Benchmark on Account Catalog (2026-07-02)

Figure 3: NVIDIA NIM model fallback chain (3a) and the tool-calling benchmark that selected the primary model (3b)

4.8 Transport, Authentication, and Delivery

The front end and backend communicate over a WebSocket that carries the goal in and streams plan updates, per-step progress, and the final answer out, giving the interface its live, animated task states. Accounts are real: passwords are scrypt-hashed in a file-backed user store, sessions persist across refreshes, and the user is greeted personally. Files written by the Writer surface as download chips in the conversation, and an optional email path can deliver the finished result as a PDF attachment.

4.9 System Workflow Summary

The complete methodology executes as a single ordered flow: goal received over WebSocket → Planner decomposes into ordered subtasks → each subtask runs in the executor's tool-calling loop with shared

memory → Synthesiser composes the Markdown answer → Critic reviews and permits at most one revision → answer and file artefacts delivered to the UI, with every LLM call along the way protected by the model-fallback chain.

5. Results and System Analysis

5.1 Behaviour of the Orchestration Pipeline

Exercised across varied goals — trip planning, tool research, workout and grocery planning, and short reports — the pipeline behaves as designed. The Planner reliably reduces a goal to a compact ordered plan, and the shared-memory design means a research subtask's findings are visibly consumed by the writing subtask that follows rather than being regenerated. The bounded tool-calling loop keeps individual specialists focused, and the artefacts left in the agent workspace confirm that outputs are real, complete content rather than the placeholder filler the shared rules explicitly forbid.

5.2 Tool Authorisation and Role Separation

The strict role-to-tool mapping is the backbone of predictable behaviour. Figure 4a shows that each specialist can reach only the tools its remit requires, with the Generalist as the sole all-access role. This scoping shrinks the model's decision space at every step and prevents, for example, a Researcher from writing files or an Analyst from performing unscoped searches.

Figure 4: Specialist × tool authorisation matrix (4a) and illustrative quality gains of the swarm over a single-agent baseline (4b)

5.3 Fault Tolerance in Practice

The model-fallback layer is what makes the system dependable rather than merely capable. In operation, a 429 on the primary model transparently advances the call to the next model in the chain, the throttled model is benched via its cooldown entry, and the user experiences continuity rather than an error. Empty completions from reasoning models — a real failure mode when reasoning consumes the whole token budget — are caught and retried on the next model, and disabling the primary's inline reasoning removes that failure mode for the majority of calls. The single-revision Critic, failing open on error, adds a quality gate that cannot itself become a point of failure.

5.4 Qualitative Comparison with a Single-Agent Baseline

Figure 4b presents an illustrative comparison between a naïve single-agent baseline and the Equilibrium swarm across four qualities: task completeness, freedom from placeholder output, success on genuinely multi-step goals, and recovery from a model outage. The swarm's advantage is largest exactly where a single call is weakest — multi-step goals and outage recovery — because those are precisely the failure modes that orchestration, shared memory, and model fallback are designed to eliminate. The figures are illustrative rather than the product of a controlled benchmark, which is noted as a limitation and a direction for future quantitative evaluation.

5.5 Strategic Implications

Taken together, the results support the core hypothesis: decomposing a goal across scoped, tool-using specialists and protecting every model call with a fallback chain produces a system that is simultaneously more capable and more robust than a monolithic prompt. The clear delineation of roles, the observable per-step progress, and the graceful degradation under model failure are the practical properties that make such a system trustworthy enough to put in front of a user.

6. Case Study: A Multi-Step Research-and-Write Goal

6.1 Scenario

Consider the goal “research the top AI agent tools of 2024 and write a structured summary to a file.” This is a canonical multi-step task: it cannot be answered from the model’s memory alone because it requires current information, and it must end in a concrete, saved artefact. It is exactly the kind of goal on which a single prompt tends to hallucinate tool names or emit a stub, and several artefacts in the agent workspace correspond to runs of this shape.

6.2 Decomposition and Routing

The Planner reduces the goal to two ordered subtasks: a research subtask assigned to the Researcher (“search the web for the top AI agent tools of 2024 and their key features”) and a writing subtask assigned to the Writer (“write a structured summary of those tools to a file using the research”). The dependency is explicit — the second subtask consumes the first’s output through shared memory — and the plan is deliberately minimal, avoiding redundant steps.

6.3 Grounded Execution

In its tool-calling loop the Researcher issues a web_search, which resolves through the search cascade (DuckDuckGo’s HTML endpoint being the keyless workhorse) and returns concrete titles and snippets. The Researcher summarises the actual findings rather than guessing. Those findings are appended to shared memory, so when the Writer runs it treats them as source material and calls write_file to save a complete, structured summary into the sandboxed workspace — the path validated to prevent escape outside the workspace directory.

6.4 Synthesis, Criticism, and Delivery

The Synthesiser composes a Markdown answer that includes the summary’s key content and names the file that was written. The Critic checks this against the goal — is it complete, on-topic, and free of placeholders — and either approves it or returns one round of targeted feedback. The finished answer streams back to the UI with the written file exposed as a download chip. Had the primary model been rate-limited at any point in this sequence, the fallback chain would have carried each call to the next available model, so the user would still have received the completed summary.

6.5 What the Case Demonstrates

This single run exercises every part of the architecture at once: decomposition, role-based routing, scoped tool use, shared memory feeding a dependent step, synthesis, criticism, tangible file output, and — latent but always present — model-level fault tolerance. It is the concrete embodiment of the claim that Equilibrium turns an unreliable single call into a coordinated, observable, and recoverable pipeline.

7. Observations

7.1 Key Findings

The implementation confirmed that an orchestrator-led swarm can decompose a plain-English goal into an appropriate ordered plan and route each subtask to a fitting specialist without human intervention. Shared memory made dependent subtasks genuinely cooperative — later agents built on earlier findings rather than repeating them — and the shared hard rules were effective at suppressing placeholder output. Scoped tool authorisation kept specialist behaviour predictable, and the synthesis-and-critic stage produced answers that were both substantive and reviewed.

The most consequential finding concerned reliability. The model-fallback layer, with its cooldown tracking and empty-response handling, transformed intermittent hosted-model failures from run-ending errors into invisible internal retries. Model-specific tuning — extra token headroom for reasoning models and disabled inline reasoning for the primary — measurably reduced empty completions and latency. Together these made the difference between a demo that works when the API is calm and a system that keeps working when it is not.

7.2 Challenges and Obstacles

Several challenges shaped the design. Hosted models are heterogeneous: some ignore tool calls, some emit corrupt arguments, and reasoning models can spend their whole budget thinking and return nothing — each of which had to be detected and worked around rather than assumed away. Coaxing strict JSON from the Planner and Critic required prompt discipline plus defensive parsing that salvages an array from surrounding prose. Confining file writes and code execution safely, given that paths and code originate from the model, demanded explicit sandboxing. Finally, the evaluation in this study is qualitative; transaction-level cost, latency distributions, and controlled success-rate benchmarking were not measured and remain the clearest limitation, motivating the quantitative work proposed next.

8. Conclusion

This project set out to solve a real weakness of naïve language-model applications — the unreliability of a single call asked to complete a complex, multi-step goal — and it did so by building Equilibrium, a full-stack multi-agent planning assistant. An orchestrator decomposes each goal into a compact ordered plan; role-specialised agents with scoped tools execute the subtasks in a shared-memory tool-calling loop; a synthesiser composes the results; and a critic enforces a single-revision quality gate. Every model call is protected by a benchmarked fallback chain with per-model cooldowns and model-specific tuning, and the whole run streams live into a React interface whose written outputs are downloadable on the spot.

The system demonstrates, concretely and reproducibly, that orchestration, scoped tool use, answer criticism, and model-level fault tolerance combine to make a language-model application both more capable and more robust than a monolithic prompt — without any model retraining. Its qualitative behaviour across diverse goals, and the case study of a research-and-write task, together validate the central hypothesis that coordinated specialisation beats a single generalist call. Equilibrium thus stands as a practical blueprint for reliable, tool-using, agentic AI systems.

9. Future Scope

Equilibrium provides a foundation on which several enhancements can be built. The most important is rigorous quantitative evaluation: controlled benchmarks of task success, answer quality, latency distributions, and per-run cost against single-agent and alternative multi-agent baselines, replacing the illustrative comparisons in this report with measured results.

- Expand the specialist set and let the orchestrator compose more elaborate plans, including parallel subtasks and conditional branches rather than a strictly linear sequence.
- Add long-term and cross-session memory so the system can learn user preferences and reuse prior research instead of starting each goal from scratch.
- Broaden the toolset — structured data connectors, calendar and email integration, richer code sandboxes, and retrieval over user documents.
- Introduce adaptive model routing that chooses a model per subtask by difficulty and cost, extending the fallback layer into a full routing policy.
- Strengthen the critic into a multi-criterion evaluator and explore bounded multi-round refinement where the quality ceiling justifies the added cost.
- Harden the system for production: concurrency, observability and tracing, rate-limit budgeting per user, and stronger sandboxing for code execution.

These directions would improve the scalability, measurable quality, and real-world applicability of the framework while preserving the orchestration-plus-resilience core that defines Equilibrium.

10. References

[1] Yao, S., et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” International Conference on Learning Representations (ICLR), 2023.

[2] Wu, Q., et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” arXiv:2308.08155, 2023.

[3] Hong, S., et al. “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework.” arXiv:2308.00352, 2023.

[4] Shinn, N., et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” Neural Information Processing Systems (NeurIPS), 2023.

[5] Schick, T., et al. “Toolformer: Language Models Can Teach Themselves to Use Tools.” NeurIPS, 2023.

[6] Anthropic. “Building Effective Agents.” Engineering guidance on orchestrator–worker and evaluator–optimizer patterns, 2024.

[7] OpenAI. “Function Calling and Tool Use.” OpenAI API Documentation, 2024.

[8] NVIDIA. “NVIDIA NIM — OpenAI-Compatible Inference Microservices.” build.nvidia.com Documentation, 2025.

[9] Patil, S. G., et al. “Gorilla: Large Language Model Connected with Massive APIs.” arXiv:2305.15334, 2023.

[10] Equilibrium (GreenLeaf) Project Source Repository — planner, roles, executor, critic, and llm modules, 2026.